

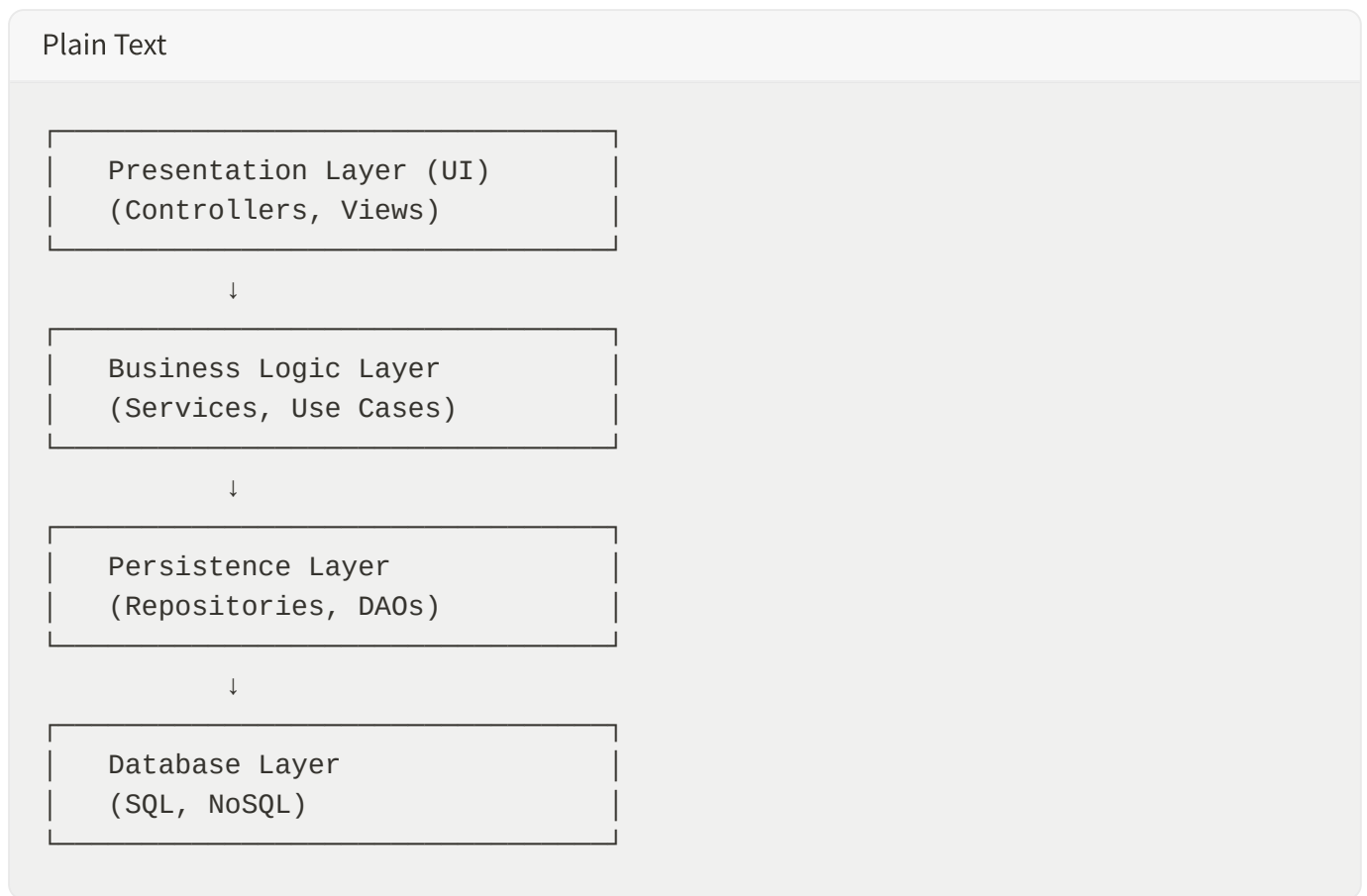
Arquiteturas Adicionais: Guia Completo

1. Arquitetura em Camadas (Layered Architecture)

1.1 Definição

A **Arquitetura em Camadas** (também chamada N-Tier Architecture) organiza a aplicação em camadas horizontais, onde cada camada tem uma responsabilidade específica. As requisições fluem de cima para baixo através das camadas.

1.2 Camadas Típicas



1.3 Características

- **Isolamento de Responsabilidades:** Cada camada tem função clara
- **Fácil de Entender:** Estrutura simples e intuitiva
- **Fácil de Testar:** Camadas podem ser testadas independentemente
- **Reutilização:** Componentes podem ser reutilizados

1.4 Prós e Contras

Prós	Contras
Simples de implementar	Monolítica por natureza
Fácil de entender	Difícil de escalar
Boa separação de responsabilidades	Acoplamento entre camadas
Testes unitários fáceis	Performance pode sofrer

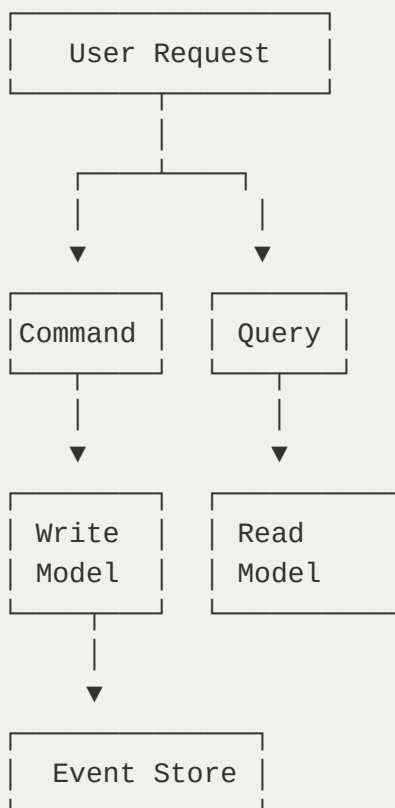
2. CQRS (Command Query Responsibility Segregation)

2.1 Definição

CQRS separa as operações de leitura (Queries) das operações de escrita (Commands). Cada uma tem seu próprio modelo de dados otimizado para seu caso de uso específico.

2.2 Componentes

Plain Text



2.3 Benefícios

- **Escalabilidade:** Escala read e write independentemente
- **Performance:** Modelos otimizados para cada operação
- **Flexibilidade:** Diferentes tecnologias para read/write
- **Auditoria:** Event store fornece histórico completo

2.4 Desafios

- **Eventual Consistency:** Dados podem estar desatualizados
 - **Complexidade:** Mais complexo de implementar
 - **Debugging:** Difícil rastrear estado
-

3. Event Sourcing

3.1 Definição

Event Sourcing armazena todas as mudanças de estado como uma sequência de eventos imutáveis. Em vez de armazenar o estado atual, armazena o histórico completo de eventos.

3.2 Fluxo

Plain Text

```
Evento 1: UserCreated(id=1, name="João")
Evento 2: UserUpdated(id=1, email="joao@email.com")
Evento 3: UserDeleted(id=1)
```

Estado Atual = Aplicar todos os eventos em sequência

3.3 Benefícios

- **Auditoria Completa:** Histórico completo de mudanças
- **Debugging:** Pode reproduzir qualquer estado anterior
- **Escalabilidade:** Fácil adicionar novos consumidores
- **Resiliência:** Recuperação de falhas é simples

3.4 Desafios

- **Eventual Consistency:** Dados podem estar desatualizados

- **Complexidade:** Requer novo paradigma de pensamento
- **Storage:** Crescimento indefinido de eventos
- **Versionamento:** Eventos podem mudar ao longo do tempo

4. Saga Pattern (Transações Distribuídas)

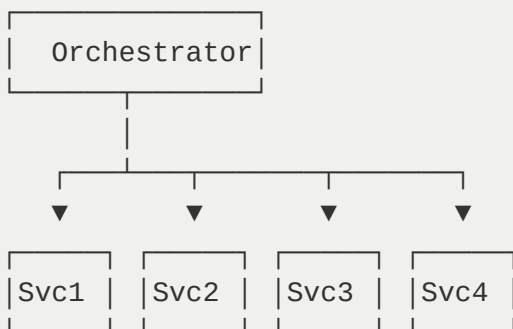
4.1 Definição

Saga Pattern é uma forma de gerenciar transações distribuídas em microserviços. Uma saga é uma sequência de transações locais que atualizam dados em diferentes serviços.

4.2 Tipos de Saga

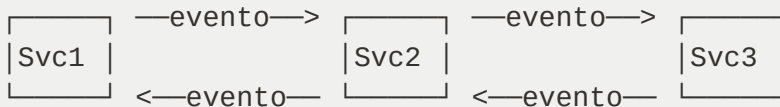
Orquestração (Orchestration)

Plain Text



Coreografia (Choreography)

Plain Text



4.3 Benefícios

- **Transações Distribuídas:** Coordena múltiplos serviços
- **Resiliência:** Pode reverter operações em caso de falha
- **Desacoplamento:** Serviços não precisam se conhecer

4.4 Desafios

- **Complexidade:** Difícil de implementar e debugar
- **Eventual Consistency:** Dados podem estar inconsistentes
- **Compensação:** Precisa definir ações de rollback

5. Padrões de Resiliência

5.1 Circuit Breaker

Previne cascata de falhas ao "abrir o circuito" quando um serviço está falhando.

Plain Text

Estados:

- CLOSED: Funcionando normalmente
- OPEN: Rejeitando requisições (serviço está falhando)
- HALF_OPEN: Testando se o serviço se recuperou

5.2 Retry Pattern

Tenta a operação novamente após uma falha temporária.

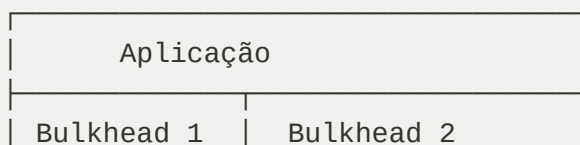
Plain Text

Tentativa 1: Falha
Espera 1s
Tentativa 2: Falha
Espera 2s
Tentativa 3: Sucesso ✓

5.3 Bulkhead Pattern

Isola recursos para evitar que uma falha afete toda a aplicação.

Plain Text



(Thread 1-5)	(Thread 6-10)
--------------	---------------

5.4 Timeout Pattern

Define tempo máximo para uma operação completar.

Plain Text

```
Requisição iniciada
Esperando resposta...
Timeout atingido (5s) → Falha
```

6. Service Mesh

6.1 Definição

Service Mesh é uma camada de infraestrutura que gerencia comunicação entre microserviços. Implementa padrões de resiliência, observabilidade e segurança.

6.2 Ferramentas Populares

- **Istio**: Mais completo e poderoso
- **Linkerd**: Leve e fácil de usar
- **Consul**: Também fornece service discovery
- **AWS App Mesh**: Gerenciado na AWS

6.3 Responsabilidades

- **Traffic Management**: Roteamento inteligente
- **Security**: Mutual TLS entre serviços
- **Observability**: Tracing e métricas
- **Resilience**: Circuit breaker, retry, timeout

7. Arquitetura Serverless

7.1 Definição

Serverless é um modelo onde você não gerencia servidores. Apenas escreve funções que são executadas sob demanda.

7.2 Provedores

- **AWS Lambda:** Função como serviço
- **Google Cloud Functions:** Alternativa do Google
- **Azure Functions:** Alternativa da Microsoft
- **OpenFaaS:** Open-source

7.3 Benefícios

- **Sem Gerenciamento de Infraestrutura:** Provedor cuida de tudo
- **Escalabilidade Automática:** Escala conforme demanda
- **Pagamento por Uso:** Paga apenas pelo que usa
- **Rápido Deploy:** Deploy em segundos

7.4 Desvantagens

- **Cold Start:** Primeira execução é mais lenta
- **Vendor Lock-in:** Difícil migrar para outro provedor
- **Limitações:** Timeout, memória, linguagens suportadas
- **Debugging:** Difícil debugar em produção

8. Arquitetura Orientada a Serviços (SOA)

8.1 Definição

SOA é um estilo arquitetural onde a aplicação é composta de serviços reutilizáveis e independentes que se comunicam através de padrões bem definidos.

8.2 Diferença entre SOA e Microserviços

SOA	Microserviços
Serviços maiores	Serviços pequenos
Compartilham dados	Dados isolados
ESB centralizado	Descentralizado

Mais complexo	Mais simples
Empresa grande	Startups/Ágil

9. Domain-Driven Design (DDD)

9.1 Definição

DDD é uma abordagem de design que coloca o domínio de negócio no centro. Alinha código com linguagem de negócio.

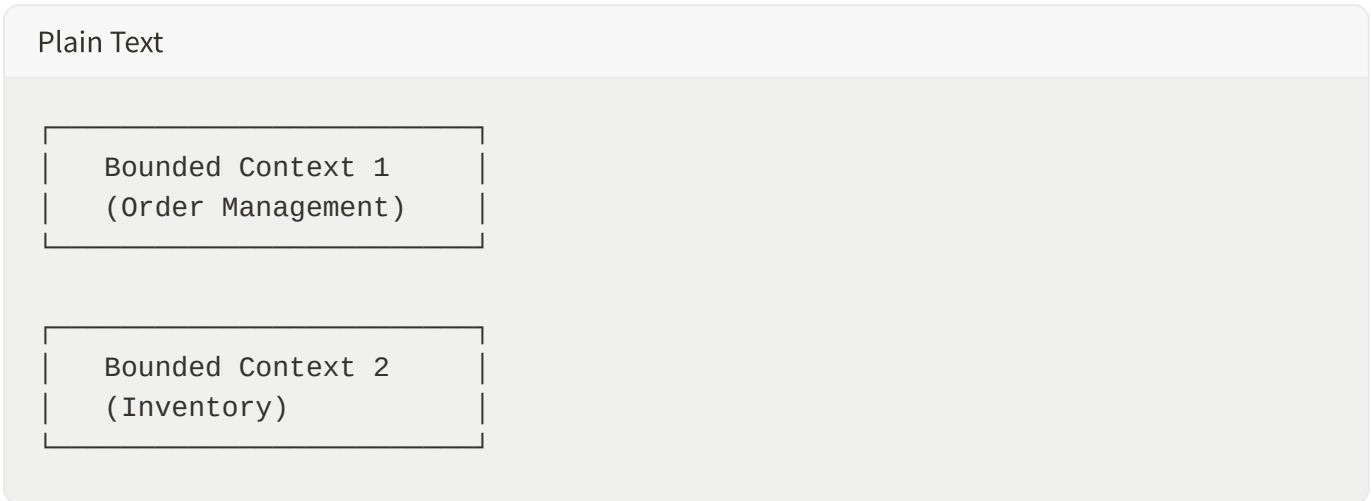
9.2 Conceitos Principais

Ubiquitous Language

Linguagem comum entre desenvolvedores e especialistas de negócio.

Bounded Context

Limite explícito onde um modelo de domínio é válido.



Aggregates

Agrupamento de objetos de domínio que são modificados juntos.

Value Objects

Objetos que representam um valor e são imutáveis.

Entities

Objetos com identidade que mudam ao longo do tempo.

9.3 Benefícios

- **Alinhamento com Negócio:** Código reflete realidade de negócio
 - **Manutenibilidade:** Fácil entender intenção do código
 - **Escalabilidade:** Estrutura natural para microserviços
 - **Qualidade:** Menos bugs, melhor design
-

10. Padrões de Deployment

10.1 Blue-Green Deployment

Dois ambientes idênticos (Blue e Green). Tráfego é alternado entre eles.

Plain Text

Versão 1 (Blue) ← Tráfego

Versão 2 (Green) ← Deploy nova versão

Teste Versão 2...

Se OK: Versão 2 (Green) ← Tráfego

Se Erro: Versão 1 (Blue) ← Tráfego (Rollback)

10.2 Canary Deployment

Nova versão é deployada para pequeno percentual de usuários.

Plain Text

Versão 1: 95% do tráfego

Versão 2: 5% do tráfego

Monitorar métricas...

Se OK: Versão 2: 100%

Se Erro: Versão 1: 100% (Rollback)

10.3 Rolling Deployment

Instâncias são atualizadas gradualmente.

Plain Text

```
Instância 1: Versão 1 → Versão 2 ✓  
Instância 2: Versão 1 → Versão 2 ✓  
Instância 3: Versão 1 → Versão 2 ✓
```

10.4 Feature Flags

Controla ativação de features sem fazer deploy.

Plain Text

```
if (featureFlags.isEnabled("new-checkout")) {  
  // Novo checkout  
} else {  
  // Checkout antigo  
}
```

11. Observabilidade

11.1 Três Pilares

Logs

Registros de eventos que aconteceram.

Plain Text

```
[2026-05-31 10:30:45] INFO: User login successful (user_id=123)  
[2026-05-31 10:30:46] ERROR: Payment processing failed (order_id=456)
```

Métricas

Medições numéricas ao longo do tempo.

Plain Text

```
- CPU Usage: 45%  
- Memory: 2.3 GB  
- Requests/sec: 1000  
- Error Rate: 0.5%
```

Tracing

Rastreamento de requisição através de múltiplos serviços.

Plain Text

```
Request → Service A (10ms)
        → Service B (25ms)
        → Service C (15ms)
Total: 50ms
```

11.2 Ferramentas

- **Prometheus:** Coleta de métricas
- **Grafana:** Visualização de métricas
- **ELK Stack:** Logging centralizado
- **Jaeger:** Distributed tracing
- **Datadog:** Observabilidade completa

12. Segurança em Arquitetura Distribuída

12.1 Autenticação

Verificar identidade do usuário.

- **JWT:** Token com informações
- **OAuth 2.0:** Delegação de acesso
- **SAML:** Single Sign-On empresarial

12.2 Autorização

Verificar permissões do usuário.

- **RBAC:** Role-Based Access Control
- **ABAC:** Attribute-Based Access Control
- **Policy-Based:** Políticas customizadas

12.3 Comunicação Segura

- **TLS/SSL:** Criptografia em trânsito
- **Mutual TLS:** Autenticação bidirecional

- **API Keys:** Chaves de acesso
- **OAuth 2.0:** Tokens de acesso

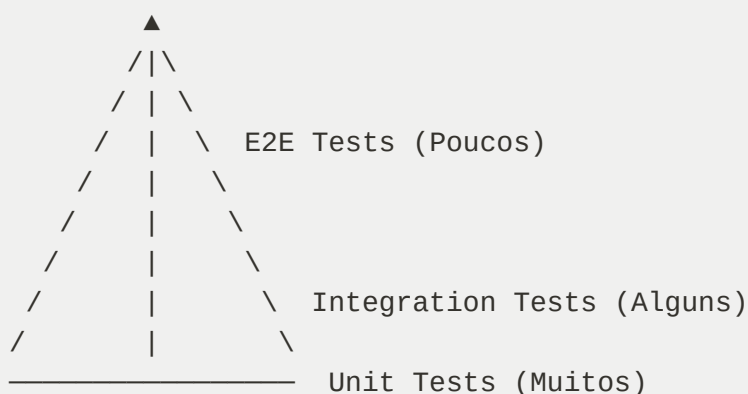
12.4 Secrets Management

- **HashiCorp Vault:** Gerenciamento centralizado
- **AWS Secrets Manager:** Gerenciado na AWS
- **Kubernetes Secrets:** Nativo do Kubernetes

13. Testes em Arquitetura Distribuída

13.1 Pirâmide de Testes

Plain Text



13.2 Tipos de Testes

Unit Tests

Testa componente isolado.

Python

```
def test_calculate_discount():  
    assert calculate_discount(100, 0.1) == 90
```

Integration Tests

Testa integração entre componentes.

Python

```
def test_order_creation_with_payment():
    order = create_order(user_id=1, items=[...])
    payment = process_payment(order_id=order.id)
    assert payment.status == "success"
```

Contract Tests

Testa contrato entre serviços.

Python

```
def test_payment_service_contract():
    # Verifica que Payment Service retorna formato esperado
    response = payment_service.process(order_id=123)
    assert "transaction_id" in response
    assert "status" in response
```

E2E Tests

Testa fluxo completo da aplicação.

Python

```
def test_complete_order_flow():
    # Login → Browse → Add to Cart → Checkout → Payment → Confirmation
```

14. Performance e Otimização

14.1 Caching

Armazena dados frequentemente acessados em memória.

- **Cache-Aside:** Aplicação verifica cache
- **Write-Through:** Escreve em cache e BD simultaneamente
- **Write-Behind:** Escreve em cache, depois em BD

14.2 Compressão

Reduz tamanho de dados em trânsito.

Plain Text

Original: 1 MB
Comprimido: 200 KB (80% redução)

14.3 CDN (Content Delivery Network)

Distribui conteúdo em servidores ao redor do mundo.

Plain Text

Usuário em São Paulo → CDN Brasil → Conteúdo rápido
Usuário em Tóquio → CDN Japão → Conteúdo rápido

14.4 Database Optimization

- **Indexação:** Acelera buscas
- **Particionamento:** Divide dados em múltiplas tabelas
- **Denormalização:** Reduz joins
- **Query Optimization:** Escreve queries eficientes

15. Migração de Monolítica para Microserviços

15.1 Estratégia Strangler Pattern

Gradualmente substitui monolítica com microserviços.

Plain Text

Fase 1: Monolítica + Novo Microserviço
API Gateway roteia requisições

Fase 2: Monolítica (reduzida) + Múltiplos Microserviços
Mais serviços extraídos

Fase 3: Apenas Microserviços
Monolítica completamente substituída

15.2 Passos

1. **Identificar Serviço:** Escolher serviço para extrair
2. **Criar Novo Serviço:** Implementar como microserviço

3. **Implementar API Gateway:** Rotear requisições
4. **Migrar Dados:** Sincronizar dados
5. **Testar:** Validar funcionamento
6. **Remover do Monolítico:** Deletar código antigo
7. **Repetir:** Próximo serviço

15.3 Desafios

- **Dados Compartilhados:** Múltiplos serviços acessam mesmos dados
 - **Transações Distribuídas:** Garantir consistência
 - **Comunicação:** Implementar chamadas entre serviços
 - **Testes:** Validar integração
-

16. Métricas DORA

16.1 Deploy Frequency

Quantas vezes a aplicação é deployada em produção.

Plain Text

```
Elite: > 1 vez por dia  
High: 1 vez por semana a 1 vez por mês  
Medium: 1 vez por mês a 1 vez por trimestre  
Low: < 1 vez por trimestre
```

16.2 Lead Time for Changes

Tempo entre commit e deploy em produção.

Plain Text

```
Elite: < 1 hora  
High: 1 hora a 1 dia  
Medium: 1 dia a 1 semana  
Low: > 1 semana
```

16.3 Mean Time to Recovery (MTTR)

Tempo para se recuperar de uma falha.

Plain Text

Elite: < 1 hora
High: 1 hora a 1 dia
Medium: 1 dia a 1 semana
Low: > 1 semana

16.4 Change Failure Rate

Percentual de deploys que causam falha.

Plain Text

Elite: 0-15%
High: 15-45%
Medium: 45-60%
Low: > 60%

17. Governança de Dados

17.1 Data Governance

Políticas e processos para gerenciar dados.

- **Data Ownership:** Quem é responsável pelos dados
- **Data Quality:** Garantir qualidade dos dados
- **Data Privacy:** Proteger dados sensíveis
- **Data Retention:** Quanto tempo manter dados

17.2 Master Data Management (MDM)

Gerencia dados mestres (clientes, produtos, etc).

Plain Text

Múltiplas Fontes → MDM → Single Source of Truth

17.3 Data Lineage

Rastreia origem e transformação dos dados.

Plain Text

18. Arquitetura Corporativa

18.1 Enterprise Architecture

Planejamento estratégico de arquitetura de toda a empresa.

- **Business Architecture:** Processos de negócio
- **Data Architecture:** Dados e informações
- **Application Architecture:** Aplicações
- **Technology Architecture:** Infraestrutura

18.2 Frameworks

- **TOGAF:** The Open Group Architecture Framework
- **Zachman:** Matriz de perspectivas
- **ArchiMate:** Linguagem de modelagem

Referências

- **Building Microservices** - Sam Newman
- **Domain-Driven Design** - Eric Evans
- **Designing Data-Intensive Applications** - Martin Kleppmann
- **Release It!** - Michael Nygard
- **The Phoenix Project** - Gene Kim

Documento preparado por: Manus AI

Data: Maio 2026

Versão: 2.0